Cloud Application Performance Monitoring and Alerting System

ROLL NO	NAME
10692373319	Shaik Athar
10692373321	Shaik Moshin
10692373333	Shaik Mukarram
10692373322	Shaik Zahed

Guided by: Mr. S. Naveen



Introduction

- Cloud applications run on distributed, elastic infrastructure
- Performance monitoring continuously collects metrics, logs and traces
- Alerting notifies teams the moment something goes wrong



Monitoring

Tracks health, speed and resource usage of apps in real time



Alerting

Triggers notifications when metrics cross defined thresholds

Why monitoring matters

Downtime hurts revenue
Every minute offline costs users and trust

Hidden failures
Distributed services fail in hidden ways

SLA compliance
Uptime and speed targets must be measured

Cost control
Right-size resources using real usage data

THE MONITORING LOOP

1 **Detect**
Spot issues via metrics

2 **Diagnose**
Trace root cause in logs

3 **Resolve**
Fix and verify recovery

Key performance metrics

Response time

How long a request takes to complete

Throughput

Requests handled per second

Error rate

Share of failed requests (4xx / 5xx)

CPU & memory

Resource usage of servers and containers

Availability

Uptime percentage measured against the SLA

Network & I/O

Bandwidth, packet loss and storage speed

System architecture



Web apps, APIs, microservices

Agents gather metrics, logs, traces

Time-series database

Evaluates rules and thresholds

Email, SMS, Slack, PagerDuty

Dashboards (Grafana, CloudWatch) read from the Metrics Store for live visualization

Three pillars of observability



Metrics

Numeric measurements sampled over time

Example: CPU 72%, latency 180 ms



Logs

Timestamped records of events and errors

Example: Payment API returned 500



Traces

End-to-end path of a request across services

Example: Login > Auth > Database

Alerting: from threshold to action

SAMPLE ALERT RULE

```
metric:     cpu_utilization
condition:  > 80%
duration:   5 minutes
action:     notify on-call
```

Tune thresholds carefully to avoid alert fatigue



Critical

Service down or error rate spike. Page the on-call engineer



Warning

Latency or CPU rising. Notify the team chat



Info

Deployments and scaling events. Log to dashboard

Popular monitoring tools

Tool	Type	Key strength
Amazon CloudWatch	AWS native	Metrics, logs and alarms for AWS services
Azure Monitor	Azure native	Application Insights, alerts, autoscale
Google Cloud Monitoring	GCP native	Dashboards, uptime checks, alerting
Prometheus + Grafana	Open source	Metric scraping, Alertmanager, dashboards
Datadog / New Relic	SaaS APM	Full-stack, multi-cloud observability

Benefits and challenges

Benefits

- Faster detection and shorter recovery time
- Better uptime and user experience
- Data-driven capacity and cost planning
- Proactive fixes using trend analysis

Challenges

- Alert fatigue from noisy thresholds
- High data volume and monitoring cost
- Visibility across microservices and multi-cloud
- Tool sprawl and integration effort

Conclusion

1. Monitoring gives continuous visibility into the health, speed and cost of cloud apps
2. Smart alerting turns raw data into fast, targeted action
3. AI-driven anomaly detection and AIOps will further reduce noise and downtime

Thank You

Questions?

Guided by

Mr. S. Naveen